LÖSUNGEN · LEHREREXEMPLAR

W0·5 – Die Methoden-Werkstatt: Lösungen

Zum Arbeitsblatt `AQ_W0_AB5_Methoden-Werkstatt.docx` · Papier-Compiler 0: Testklasse `PapierCompiler0Test` in `AQ_W0_Akademie_Geruest.txt` · alle Lösungen und Werte mit Java (javac 21) geprüft, die Fehlergalerie-Beispiele ebenso

Vorgemachtes Beispiel – Kontrolle

laengsteSerie liefert (x = 3): {3, 3, 1, 3, 3, 3, 2} → 3 · {} → 0 · {1, 2} → 0 · {3} → 1 · {1, 3, 3} → 2. Die Fehlvariante (Rekord nur im else-Zweig aktualisiert) liefert für {1, 3, 3} und {3} jeweils **0** – mit Java nachgeprüft; ein gutes Beispiel für den Wert von Schritt 3.

Fehlergalerie – geprüfte Werte

F3: `if (gefunden = true)` mit gefunden = false liefert trotzdem "ja" (true). F4: 0.0 statt 75.0. F5: summeAb({5, 1, 2}, 1) liefert 8 statt 3. F6: allePositiv({4, −1, 3}) liefert true statt false. F7: nach negiertFalsch({2, −5}) ist auch r = {-2, 5}. F9: "".charAt(0) → StringIndexOutOfBoundsException. F10: {−4, −2, −9} liefert 0 statt −2. F1 wird vom Compiler abgelehnt („missing return statement“), F8 kompiliert.

Aufgabe 1 – Fehler-Bingo

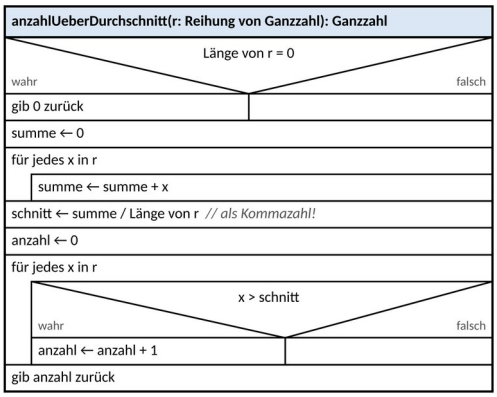
Nr.	F-Nr.	Korrektur
S1	F2	Start bei `i = 0` (sonst fehlt r[0]; für {4, 7, 1} entsteht 8 statt 12)
S2	F8	`if (name.equals("Aria"))`
S3	F4	`double prozent = 100.0 * richtige / gesamt;` (falsch: 0.0, richtig 87.5)
S4	F6	else-Zweig streichen; `return false` nur nach der Schleife
S5	F1	zweites if durch else ersetzen bzw. `return b;` am Ende (Gleichstand a == b hat keinen Pfad)
S6	F10	Startwert `r[0]` (nach Leer-Abfrage); für {5, 3, 8} entsteht sonst 0 statt 3
S7	F7	`int[] kopie = new int[original.length];` und Werte einzeln kopieren
S8	F3	`if (!fertig)` bzw. `if (fertig == false)` – = weist zu, Bedingung immer falsch
S9	F9	`if (r.length == 0) { return -1; }` vor dem Zugriff
S10	F5	Zeile `anzahl = 3;` streichen

Aufgabe 2 – Papier-Compiler 0 (5 BE)

a) `public static int anzahlUeberDurchschnitt(int[] r)` · Sonderfälle: leere Reihung → 0 (keine Division durch 0), **echt** größer, Durchschnitt als **Kommazahl**.

r	{12, 7, 15, 9, 3}	{5, 5, 5}	{ }	{-1, -2}	{7}
Durchschnitt	9.2	5.0	–	−1.5	7.0
erwartet	2	0	0	1	0

```java
public static int anzahlUeberDurchschnitt(int[] r) {
    if (r.length == 0) {
        return 0;
    }
    int summe = 0;
    for (int x : r) {
        summe = summe + x;
    }
    double schnitt = (double) summe / r.length;
    int anzahl = 0;
    for (int x : r) {
        if (x > schnitt) {
            anzahl++;
        }
    }
    return anzahl;
}
```



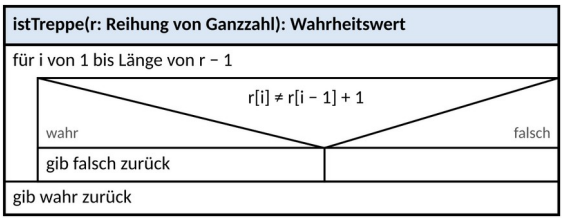
Kernpunkt	BE
Methodenkopf wie in der Signatur	1
Leer-Abfrage vor der Division mit Rückgabe 0	1
Summe über **alle** Elemente (Index 0 bis length − 1)	1
Durchschnitt als Kommazahl (Cast vor der Division oder Kommazahl-Konstante)	1
zweiter Durchlauf zählt Werte mit **echt** größer; Rückgabe der Anzahl	1

d) Der Durchschnitt von {−1, −2} ist **−1.5**; die Lösung liefert 1. Mit ganzzahliger Division wäre er −3 / 2 = **−1** und das Ergebnis 0 (Test `durchschnittIstKommazahl` rot). Bei positiven Werten fällt die ganzzahlige Division in diesem Test **nicht** auf (zwischen abgerundetem und echtem Durchschnitt liegt keine Ganzzahl) – trotzdem im Abitur Abzug. Weitere typische Fehler (vgl. LSG-Datei): `>=` statt `>` (Test `echtGroesser`), Division durch `r.length - 1`, return in der Schleife. **Achtung:** Ohne Leer-Abfrage entsteht bei Kommazahl-Division 0.0 / 0 = NaN, jeder Vergleich ist false – der Test `leereReihung` wird trotzdem grün. Im Abitur kostet die fehlende Abfrage den Kernpunkt → beim Papier-Compiler ansprechen.

e) Hauptprogramm nach dem Freischalten: „Ueber dem Durchschnitt: 2 Lehrlinge (erwartet: 2)“.

Aufgabe 3 – istTreppe (4 BE)

```java
public static boolean istTreppe(int[] r) {
    for (int i = 1; i < r.length; i++) {
        if (r[i] != r[i - 1] + 1) {
            return false;
        }
    }
    return true;
}
```



Geprüft: {4, 5, 6, 7} → true · {4, 5, 7} → false · {} → true · {9} → true · {3, 2} → false · {−2, −1, 0} → true · {5, 6, 6} → false. **Kernpunkte:** Kopf (1) · Schleife ab i = 1 bzw. bis length − 2 mit `r[i + 1]` – kein Indexfehler (1) · Abbruch mit `false` beim ersten Verstoß (1) · `true` erst nach der Schleife, deckt Länge 0 und 1 ohne Sonderfall ab (1). Typischer Fehler: F6 (`else return true` in der Schleife).

Aufgabe 4 – anzahlMit mit Hilfsmethode (5 BE)

```
public static boolean enthaeltZeichen(String s, char c) {
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == c) {
            return true;
        }
    }
    return false;
}
// gleichwertig (erlaubte Operationen): return s.contains("" + c);

public static int anzahlMit(String[] woerter, char c) {
    int anzahl = 0;
    for (String w : woerter) {
        if (enthaeltZeichen(w, c)) {
            anzahl++;
        }
    }
    return anzahl;
}
```

Geprüft mit {"Drache", "Elf", "Ork", "Zwerg"}: c = 'r' → 3, 'E' → 1, 'e' → 2; leere Reihung → 0. **Kernpunkte:** beide Köpfe korrekt (1) · enthaeltZeichen: Zeichenvergleich mit `==` auf `charAt` (oder contains), `return false` erst nach der Schleife (2) · anzahlMit: Zähler über alle Wörter (1) · Aufruf der Hilfsmethode mit Argumenten in richtiger Reihenfolge, Rückgabe (1). Nicht erlaubt: `indexOf`, `toLowerCase`.

Aufgabe 5 – mitte (4 BE)

```
public static int[] mitte(int[] r) {
    if (r.length < 3) {
        return new int[0];
    }
    int[] erg = new int[r.length - 2];
    for (int i = 0; i < erg.length; i++) {
        erg[i] = r[i + 1];
    }
    return erg;
}
```

Geprüft: {3, 8, 1, 9} → {8, 1} · {3, 8} → {} · {} → {} · {1, 2, 3} → {2} · {4, 5, 6, 7, 8} → {5, 6, 7}; r bleibt {3, 8, 1, 9}. **Kernpunkte:** Kopf mit `int[]` (1) · Sonderfall: leere Reihung, **nicht** null – ohne Abfrage entstünde bei Länge 0 oder 1 `new int[-2]` bzw. `new int[-1]` → Laufzeitfehler (1) · neue Reihung der Länge r.length – 2 (1) · Kopieren mit Verschiebung `r[i + 1]` und Rückgabe (1).

Folie für die Peer-Korrektur (Aufgabe 6) – nach der Schreibphase zeigen

Aufgabe	Kernpunkt 1	Kernpunkt 2	Kernpunkt 3
3 istTreppe	Vergleich mit dem Vorgänger `r[i - 1] + 1` ohne Indexfehler	`false` sofort beim ersten Verstoß	`true` erst nach der Schleife (Länge 0 und 1 inklusive)
4 anzahlMit	enthaeltZeichen: `return false` erst nach der Schleife	anzahlMit ruft enthaeltZeichen auf (nicht neu programmiert)	Zeichen mit `==`, keine verbotenen Operationen
5 mitte	Sonderfall Länge < 3 → `new int[0]`, nicht null	neue Reihung der Länge r.length – 2	`erg[i] = r[i + 1]` – Verschiebung um 1, r unverändert

Bewertungsvorschlag: Kernpunkte zuerst, dann die allgemeinen Zeilen des Bogens (Kopf, Pfade, Grenzen). Hinweise von Helfer:innen, die zu einer Korrektur geführt haben, im Plenum würdigen (Rückmeldung zeigen lassen) – ohne Punkte. Häufige Fehler aus den Fehlerlogbüchern als Warm-up-Wiederholung vor Boss 0 aufgreifen (Warm-up W0-21, W0-22).